FinanceBot

AI-Powered Personal Finance Advisor

COMPLETE PROJECT DOCUMENTATION

Project Detail	Information
Project Domain	FinTech / AI-Assisted Financial Planning
Backend	Python 3.10+ · Flask 3.0.3
AI Service	Google Gemini 2.0 Flash (Gemini 1.5 Flash fallback)
Frontend	HTML5 · CSS3 · Vanilla JavaScript
Fallback Mode	Built-in Smart Advisory Engine (50/30/20 framework)
Author	Kunal Khyade
Repository	github.com/kunalkhyade-create/-New-Personal-Finnace-Advisor-Bot-
Documentation Date	October 2026

AI-powered budgeting, category-wise spending analysis, and goal-oriented saving guidance.

Prepared using the project source code and documentation in the GitHub repository, following the structure and level of detail of the supplied reference PDF.

Contents

1. Project Information
2. Abstract

3. Introduction and Problem Statement
4. Objectives

5. Scope of the Project
6. Key Features

7. Technology Stack
8. System Architecture

9. Application Workflow
10. Project Directory Structure

11. Backend Design and Core Functions
12. Flask Routes and API Overview

13. Input Validation and Financial Data
14. AI Engine and Prompt Engineering

15. Smart Advisory Fallback Engine
16. Frontend Design and Interaction

17. Data Flow and Response Structure
18. Example Input and Budget Calculation

19. Error Handling
20. Security and Privacy

21. Installation and Execution
22. User Guide

23. Testing Checklist
24. Deployment

25. Advantages
26. Limitations

27. Future Scope
28. Educational Value

29. Technical Architecture Summary
30. Conclusion and References

Note: This documentation describes the repository implementation. Items in the testing checklist are verification steps, not claims that every test has been executed.

1. Project Information

Field	Details
Project Name	FinanceBot — AI Personal Finance Advisor
Project Type	AI-powered, web-based personal finance advisory application
Primary Purpose	Generate budget guidance, analyse spending categories, and suggest savings actions from user-entered financial data
Backend	Python 3.10+ with Flask 3.0.3
AI Integration	Google Gemini 2.0 Flash; Gemini 1.5 Flash is used as an automatic model fallback
Offline / No-key Mode	Custom Smart Advisory Engine based on the 50/30/20 budgeting framework and Indian household benchmarks
Frontend	HTML5, CSS3, Vanilla JavaScript (ES2022+)
Data Exchange	HTTP requests and JSON responses
Database	No persistent database; request data is processed in memory
Version Control	Git and GitHub
Public Repository	https://github.com/kunalkhyade-create/-New-Personal-Finnace-Advisor-Bot-
Live Demo Link in README	https://kunalkhyade-create.github.io/-New-Personal-Finnace-Advisor-Bot-/

The project is designed as a lightweight single-page application. The user provides income, expense amounts, and a financial goal; the server returns a structured analysis for display in the browser.

2. Abstract

FinanceBot is a web-based personal finance advisor that helps users understand their monthly financial position. It accepts monthly take-home income, expense values across supported categories, and a selected financial goal. The Flask backend validates the submitted information and produces a structured response containing a recommended budget, category-level spending analysis, saving suggestions, and a financial summary.

The application supports two analysis paths. When a valid Google Gemini API key is available, the backend builds a financial prompt and requests a response from Gemini 2.0 Flash, with Gemini 1.5 Flash as a model fallback. When no key is configured, or the Gemini service cannot be used, the built-in Smart Advisory Engine generates a result using rule-based calculations and budgeting benchmarks.

The frontend uses JavaScript fetch requests to communicate with Flask and dynamically displays the returned information without requiring a full page reload. The current implementation is stateless and does not use a persistent database or user account system.

The project demonstrates practical integration of a generative AI service, a deterministic fallback engine, REST-style routes, JSON processing, validation, and responsive browser-based presentation.

3. Introduction and Problem Statement

People may know their income and expenses but still find it difficult to identify overspending, decide how much to allocate to different needs, or connect monthly savings with a specific goal. Spreadsheet-based tracking can also require manual interpretation.

Problem addressed: How can a simple browser-based tool convert user-entered income and expense information into understandable budget guidance and actionable saving suggestions, without requiring a paid advisory service or a complex setup?

FinanceBot addresses this by providing a single-page interface and two analysis modes: live AI-assisted analysis and a built-in algorithmic mode that works without an API key.

4. Objectives

- Accept a user's monthly take-home income and expense details.
- Support multiple common expense categories and eight financial goals.
- Validate financial inputs on the server before analysis.
- Integrate Google Gemini for AI-generated budgeting guidance.
- Provide a built-in Smart Advisory Engine when Gemini is not configured or fails.
- Generate category-level recommended and actual spending comparisons.
- Provide rupee-quantified saving suggestions and an overall summary.
- Display results dynamically through a browser-based interface.
- Allow runtime Gemini API key configuration through the interface.
- Provide a lightweight project that can be run locally and shared through a temporary Ngrok tunnel.

5. Scope of the Project

Included in the current implementation

- Monthly income entry in Indian Rupees (INR).
- Eleven supported expense categories.
- Eight selectable financial goals.
- Gemini-based analysis and automatic model fallback.
- Offline Smart Advisory Engine based on budget rules.
- Budget cards, category analysis, saving suggestions, and financial summary.
- API key status and configuration interface.
- Server-side validation, JSON API responses, and error handling.
- Local Flask execution and optional public sharing through Ngrok.

Outside the current implementation

- Direct bank-account connection or automatic transaction import.
- Persistent storage of income, expenses, goals, or analysis history.
- User registration, login, or multi-user financial profiles.
- Automatic payments, transfers, or investment transactions.
- Guaranteed investment returns or professional financial advice.
- Full accounting, tax filing, or regulated financial advisory functionality.

6. Key Features

Feature	Description
AI Budget Generator	Creates a monthly budget plan using Gemini 2.0 Flash when configured.
Spending Analyser	Compares category-wise actual spending with recommended allocations and status indicators.
Saving Suggestions	Generates approximately 4–6 practical suggestions with estimated rupee savings, depending on the input.
Financial Summary	Provides a short overall description of the user's financial position.
Smart Advisory Engine	Produces results without an external AI key using built-in rules and the 50/30/20 framework.
Automatic Gemini Model Fallback	Attempts Gemini 1.5 Flash if the primary Gemini 2.0 Flash model cannot be used.
In-App API Key Manager	Allows a user to activate or clear a Gemini key while the application is running.
Input Validation	Checks submitted income and expense data on the server.
Public Sharing	Includes an Ngrok launcher for temporary access to the local Flask app.

7. Technology Stack

Technology	Role in the Project
Python 3.10+	Backend language and application logic.
Flask 3.0.3	HTTP server, routing, request handling, and template rendering.
Google Gemini API	Generative AI analysis through Gemini 2.0 Flash and Gemini 1.5 Flash fallback.
HTML5 / Jinja2	Page structure and server-rendered template.
CSS3	Dark glassmorphism design system, responsive layout, cards, and progress styling.
Vanilla JavaScript	Form handling, fetch requests, API-key interface, and dynamic result rendering.
JSON	Request and response data format between browser and backend.
python-dotenv	Loads configuration from environment variables and .env.
pyngrok	Creates a temporary public tunnel to the local Flask server.
Gunicorn	Included as a deployment-related dependency.
Git / GitHub	Source control, collaboration, and project hosting.

The dependency versions listed in the repository's requirements.txt are Flask 3.0.3, google-generativeai 0.8.3, python-dotenv 1.0.1, pyngrok 7.2.0, and gunicorn 21.2.0.

8. System Architecture

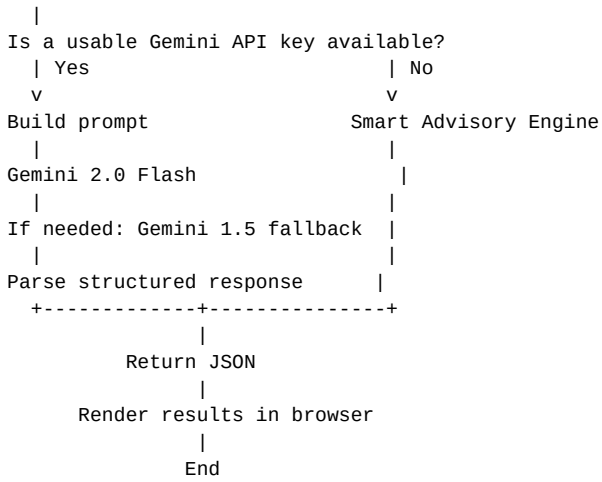
FinanceBot follows a stateless single-page web application pattern. The browser sends the complete financial input in a JSON request. Flask validates the data, chooses the Gemini or Smart Advisory path, and returns a JSON object. JavaScript then renders the returned sections.

```
USER
  |
  v
BROWSER CLIENT
  templates/index.html + static/css/style.css + static/js/main.js
  |
  | POST /analyse (JSON)
  v
FLASK BACKEND (app.py)
  |
  +--> validate_input()
  |
  +--> get_active_api_key()
         |
         +--> Key available
         |      |
         |      v
         |   build_prompt()
         |      |
         |      v
         |   Gemini 2.0 Flash
         |      |
         |      +--> Gemini 1.5 Flash fallback
         |      |
         |      v
         |   extract_json()
         |
         +--> No key / AI failure
                |
                v
         generate_smart_fallback()
  |
  v
STRUCTURED JSON RESPONSE
  |
  v
JAVASCRIPT RESULT RENDERING
  Budget | Analysis | Suggestions | Summary
```

The architecture separates browser presentation, backend validation and orchestration, and the external AI service. The fallback engine provides a second route to a usable result when live AI is unavailable.

9. Application Workflow

```
Start
  |
Open FinanceBot in a browser
  |
Enter monthly income
  |
Enter applicable expense amounts
  |
Choose a financial goal
  |
Submit the form
  |
JavaScript collects values and sends JSON
  |
POST /analyse
  |
Flask validates the request
```


Is a usable Gemini API key available?
Yes
No
Build prompt
Smart Advisory Engine
Gemini 2.0 Flash
If needed: Gemini 1.5 fallback
Parse structured response
Return JSON
Render results in browser
End

10. Project Directory Structure

```
-New-Personal-Finnace-Advisor-Bot-/
|
|-- app.py                    # Flask routes, validation, AI and fallback logic
|-- run_public.py             # Ngrok public tunnel launcher
|-- requirements.txt          # Python dependencies
|-- .env.example              # Environment-variable template
|-- .gitignore                # Excludes local and sensitive files
|-- Procfile                  # Process entry configuration
|-- README.md                 # Project overview and quick start
|-- REPORT.md                 # Detailed project report
|-- CONTRIBUTING.md           # Contribution guidance
|
|-- index.html                # Root HTML file (repository's static/demo entry)
|
|-- templates/
|   |-- index.html            # Flask/Jinja2 application template
|
|-- static/
|   |-- css/
|   |   |-- style.css         # Visual design and responsive styling
|   |
|   |-- js/
|       |-- main.js           # Client-side application behavior
|
|-- docs/
    |-- architecture.md       # Architecture details
    |-- database.md           # Stateless data model description
    |-- API_Documentation.md   # API reference
    |-- USER_GUIDE.md         # User instructions
```

The repository contains both a root index.html and a Flask template at templates/index.html. The Flask route documented in the source renders the template in the templates directory.

11. Backend Design and Core Functions

The main backend implementation is app.py. It creates the Flask application, defines route handlers, validates submitted values, selects the AI mode, and formats the response.

Function / Component	Responsibility
get_active_api_key(client_key)	Resolves a usable Gemini key from the client request or configured environment values.
build_prompt(income, expenses, goal)	Builds the instruction prompt using the user's financial information and required structured output.
extract_json(raw)	Extracts and parses JSON from the AI response, including responses wrapped in Markdown fences.
generate_smart_fallback(income, expenses, goal)	Computes budget recommendations, category status, saving suggestions, and summary without an external AI call.
validate_input(data)	Checks incoming data types, required values, allowed categories, and numeric bounds.
index()	Renders the main application page with category and goal information.
analyse()	Processes the submitted financial data and returns analysis as JSON.
key_status()	Reports or updates Gemini API-key configuration.
health()	Returns a lightweight service health response.

Backend responsibilities

- Start the Flask application and load configuration.
- Render the web interface.
- Receive JSON requests from the browser.
- Validate income, expenses, and goal values.
- Choose live Gemini analysis or the Smart Advisory Engine.
- Parse the AI result and return a consistent response.
- Handle invalid requests and AI-service errors.

12. Flask Routes and API Overview

Route	Method	Purpose
/	GET	Renders the FinanceBot web application.
/analyse	POST	Validates financial input and returns budget, analysis, suggestions, summary, and source information.
/api/key-status	GET	Returns whether Gemini mode is configured / active.
/api/key-status	POST	Accepts a Gemini API key for runtime configuration.
/health	GET	Provides a lightweight health-check response.

POST /analyse — Main request

The endpoint expects JSON. The income field is required and must be greater than zero and no more than ■10,000,000. The expenses field is an object containing at least one supported category. The goal is optional and defaults to emergency_fund when omitted.

```
{
  "income": 75000,
  "expenses": {
    "rent": 20000,
    "food": 8000,
    "transport": 3000,
    "dining": 4000,
    "entertainment": 2000,
    "utilities": 2500,
    "savings": 10000
  },
  "goal": "emergency_fund"
}
```

A successful response contains success, source, budget, analysis, suggestions, and summary fields. Gemini mode may also identify the model used. The source field distinguishes Gemini output from Smart Advisory Engine output.

Authentication: the application does not require a user login. The Gemini API key is optional; without a usable key, the built-in engine is selected.

13. Input Validation and Financial Data

Supported expense categories

Key	Category	Guideline used by the project
rent	Rent / Housing	30%
food	Groceries and Food	15%
transport	Transport	10%
dining	Dining Out	8%
entertainment	Entertainment	8%
utilities	Utilities	8%
savings	Savings	20% target
health	Health and Medical	5%
education	Education	10%
shopping	Shopping	8%
other	Miscellaneous	10%

Supported financial goals

Goal key	Purpose
emergency_fund	Build a six-month emergency fund.
vacation	Save for a planned vacation.
gadget	Save for a major gadget purchase.
investment	Begin a disciplined investment portfolio.
debt_repayment	Pay down existing debt or credit-card balances.
home_purchase	Save toward a home down payment.
retirement	Build long-term retirement savings.
education	Save for higher education or professional certifications.

Validation rules documented in the repository

- Monthly income must be numeric, greater than zero, and at most ■10,000,000.
- Expense values must be valid non-negative numeric amounts and are checked against the configured per-category upper bound.
- Only the supported expense keys are accepted by the validation layer; unknown keys are ignored.
- The request must contain usable JSON data and at least one supported expense category.

14. AI Engine and Prompt Engineering

The live AI path uses Google's Gemini API. The backend constructs a prompt from the submitted income, expense categories, and selected goal. It requests a structured response so the frontend can display the output in predictable sections.

Gemini model selection

- Primary model: Gemini 2.0 Flash.
- Automatic model fallback: Gemini 1.5 Flash if the primary model cannot be used.
- The API key may be supplied through the supported client request mechanism or server configuration.
- If no usable key is available, the application can still generate a result through its built-in engine.

Prompt inputs

- Monthly take-home income.
- Expense amounts for supported categories.
- Selected financial goal.
- Instructions for analysis and recommendations.
- Required structured JSON response format.

Response parsing

The extract_json() helper attempts to handle model output that may contain JSON directly or JSON wrapped in Markdown fences. The backend parses the response before returning it to the browser. This step helps separate the response data from explanatory formatting.

```
{
  "success": true,
  "source": "gemini",
  "budget": [],
  "analysis": [],
  "suggestions": [],
  "summary": "..."
}
```

AI output can vary and should be treated as educational guidance rather than guaranteed financial advice.

15. Smart Advisory Fallback Engine

The Smart Advisory Engine is a built-in algorithmic path. It allows the application to operate without a Gemini API key and acts as a fallback when the Gemini request fails.

Main approach

- Uses a 50/30/20 budgeting framework as a general reference.
- Uses category-specific recommended percentages and Indian household spending benchmarks defined in the backend.
- Compares actual category values with recommended values.
- Assigns a status such as Over, On-Track, or Under.
- Generates practical suggestions based on the expense values and the selected goal.
- Returns a summary and structured data compatible with the frontend.

This is a deterministic, rule-based engine rather than a generative AI model. It does not require a network call to Gemini to calculate its guidance.

Why two engines?

The dual-engine design improves availability for demonstrations and local use. Users can test the core workflow even when an API key has not been configured or an external AI service is temporarily unavailable.

16. Frontend Design and Interaction

The frontend is implemented with a Jinja2 HTML template, a CSS design system, and Vanilla JavaScript. The repository describes the visual style as dark glassmorphism with a blue-teal palette.

File	Responsibility
templates/index.html	Contains the application layout, financial profile form, result containers, and API-key modal.
static/css/style.css	Defines colours, typography, layout, cards, responsive grid, and animated progress styling.
static/js/main.js	Collects form values, submits asynchronous requests, manages API-key interactions, and renders returned results.

Main interface sections

- Header with FinanceBot identity and AI status / API-key control.
- Hero section explaining the budgeting and analysis purpose.
- Financial profile form for monthly income and expense categories.
- Financial goal selector.
- Analysis action and feedback/loading states.
- Results area containing budget cards, category analysis, suggestions, and financial summary.

Dynamic rendering

JavaScript uses the Fetch API to send data to /analyse. When the server responds, the browser updates the results section without a full page reload. The frontend includes functions for rendering budget data, analysis items, and saving suggestions. Dynamic text is escaped before insertion to reduce cross-site scripting risk.

17. Data Flow and Response Structure

```
USER INPUT
  income + expenses + goal
        |
        v
JavaScript collects form data
        |
        v
JSON request to Flask
        |
        v
Server-side validation
        |
        v
Gemini analysis OR Smart Advisory Engine
        |
        v
Structured JSON response
        |
        v
Frontend renders:
  1. Budget recommendations
  2. Spending analysis
  3. Saving suggestions
  4. Financial summary
```

Response sections

Field	Purpose
success	Indicates whether the request was processed successfully.
source	Identifies the selected engine, such as gemini or smart_engine.
budget	List of categories with recommended and actual values, percentage, status, and tip.
analysis	Category-level insight and spending status.
suggestions	List of saving actions, often with estimated rupee amounts.
summary	Overall financial position summary.
model	Model identifier when Gemini mode returns it.

The precise response values depend on the user's request and the engine that generated the result.

18. Example Input and Budget Calculation

The following is an illustrative request based on the API documentation. It is an example for explaining the data flow, not a recorded user result.

Input	Amount
Monthly income	■75,000
Rent / Housing	■20,000
Food	■8,000
Transport	■3,000
Dining Out	■4,000
Entertainment	■2,000
Utilities	■2,500
Savings	■10,000

The listed expenses including planned savings total ■49,500. The remaining amount from these listed figures is ■25,500.

```
Total listed amounts =
20,000 + 8,000 + 3,000 + 4,000
+ 2,000 + 2,500 + 10,000
= Rs. 49,500

Remaining amount =
75,000 - 49,500
= Rs. 25,500
```

The application passes the user's income, expense values, and goal to the selected analysis engine. The returned budget recommendations and explanations are then shown in the browser.

19. Error Handling

Situation	Expected handling described by the implementation
Invalid or missing input	Validation rejects unusable values and returns an error response.
No Gemini API key	The Smart Advisory Engine is selected automatically.
Gemini model failure	The backend can attempt the alternate Gemini model and use the Smart Engine if live AI remains unavailable.
Malformed AI response	The backend attempts to extract and parse JSON before returning a structured result.
API key configuration issue	The interface provides feedback and the application can continue in Smart Engine mode.
Server health check	The /health route provides a lightweight status response.

The purpose of error handling is to avoid making the application dependent on a successful external AI request for every analysis.

20. Security and Privacy

- Environment variables are used for server-side secrets; .env.example provides a template.
- The .gitignore file is intended to keep local secret/configuration files out of version control.
- The README instructs users not to commit real API keys.
- Server-side validation checks submitted financial data before analysis.
- Frontend dynamic strings are escaped before being inserted into the page.
- The application is stateless: financial profile data is processed for the request and no persistent database is used in the current version.

Important deployment note: Use a private, properly managed secret configuration for production. Do not publish Gemini or Ngrok credentials in source code, screenshots, README files, or commits. The current runtime key manager and public-tunnel launcher should be reviewed before exposing the application to untrusted users.

FinanceBot is an educational budgeting aid. It is not a substitute for professional financial, investment, tax, or legal advice.

21. Installation and Execution

Prerequisites

- Python 3.10 or later.
- pip package installer.
- A modern web browser.
- Optional: Google Gemini API key for live AI mode.

Step 1 — Clone the repository

```
git clone https://github.com/kunalkhyade-create/-New-Personal-Finnace-Advisor-Bot-.git
cd -New-Personal-Finnace-Advisor-Bot-
```

Step 2 — Create and activate a virtual environment

```
# Windows PowerShell
python -m venv myenv
myenv\Scripts\activate

# macOS / Linux
python3 -m venv myenv
source myenv/bin/activate
```

Step 3 — Install dependencies

```
pip install -r requirements.txt
```

Step 4 — Configure environment variables

```
# Windows
copy .env.example .env

# macOS / Linux
cp .env.example .env
```

Add your own values to .env if you want live Gemini analysis:

```
GEMINI_API_KEY=your_gemini_api_key_here
SECRET_KEY=replace_with_a_long_random_secret
```

Step 5 — Run the application

```
python app.py
```

Step 6 — Open the application

```
http://127.0.0.1:5000
```

> The application can run without a Gemini key using the Smart Advisory Engine.

22. User Guide

Step	User action
1	Open FinanceBot in a browser at the local application URL.
2	Enter monthly take-home income in Indian Rupees.
3	Enter amounts for the expense categories that apply. Unused categories may be left blank.
4	Select a financial goal, such as emergency fund, vacation, gadget, investment, debt repayment, home purchase, retirement, or education.
5	Submit the form to start analysis.
6	Review the budget cards and compare recommended values with actual spending.
7	Read category-level insights and status indicators.
8	Review saving suggestions and the overall financial summary.

Choosing the analysis mode

The Smart Advisory Engine works without an API key. To use Gemini, configure a valid key using the API-key control in the application or through server environment configuration. The interface displays the active mode. Clearing the configured key returns the application to the built-in engine.

Understanding the results

- Budget: recommended allocation compared with the entered actual amount.
- Analysis: category-specific spending insight and status.
- Suggestions: practical actions that may help reduce spending or support the selected goal.
- Summary: a concise overview of the submitted financial profile.

23. Testing Checklist

Use this checklist when verifying a local installation. The checklist is provided as a test plan; individual items should be marked only after they have been tested.

Area	Verification step
Application launch	The Flask server starts and the home page opens.
Input form	Income and supported expense fields accept valid values.
Goal selection	Each supported financial goal can be selected.
Analysis request	Submitting the form sends JSON to POST /analyse.
Gemini mode	A valid Gemini key produces a Gemini-sourced response.
Fallback mode	No API key produces a Smart Engine response.
Model fallback	A primary Gemini model error is handled by the alternate model path.
Results	Budget, analysis, suggestions, and summary appear in the interface.
Validation	Invalid income or expense values produce a useful error.
API key controls	The key status can be checked, activated, and cleared.
Responsive UI	The page remains usable on desktop and mobile screen sizes.
Security	Real secrets are absent from committed source and public documentation.

24. Deployment

Local development

The standard development command is python app.py. The Flask application runs locally and is accessed through the browser at http://127.0.0.1:5000.

Temporary public sharing with Ngrok

The repository includes run_public.py, which starts a tunnel to port 5000 using pyngrok and starts the Flask application in a background thread. It prints the generated public URL for sharing.

```
python run_public.py
```

Before using this launcher, configure a valid Ngrok authentication token securely. Do not commit a real token into run_public.py. For production, use a proper hosting platform, HTTPS, managed secrets, and a production WSGI server rather than relying on a development server or temporary tunnel.

The repository README also lists a GitHub Pages demo URL. GitHub Pages serves static files; the full analysis workflow described in this document depends on the Flask backend routes. A working public deployment therefore requires the backend to be reachable as well.

25. Advantages

- Simple browser-based workflow for entering income and expenses.
- Two-engine design: Gemini AI plus a built-in no-key fallback.
- Category-level analysis rather than only a single total.
- Goal-oriented saving suggestions.
- Structured JSON communication between frontend and backend.
- No database setup is required for the current stateless workflow.
- Can be run locally with a small Python dependency set.
- Demonstrates AI API integration and prompt engineering.

26. Limitations

- No persistent database or financial history is maintained.
- No user authentication or separate user profiles are provided.
- No automatic bank or transaction integration is available.
- AI-generated suggestions may be incomplete or inaccurate.
- Rule-based recommendations are based on fixed guidelines and may not fit every household.
- The application does not guarantee savings, investment performance, or financial outcomes.
- Live Gemini analysis depends on API availability, key validity, and provider limits.

27. Future Scope

- Add optional user accounts and secure authentication.
- Introduce an encrypted database for users who want to save financial history.
- Build monthly and yearly spending dashboards with charts.
- Add goal-progress tracking and target dates.
- Support PDF and CSV report exports.
- Allow users to compare budget versus actual spending across months.
- Improve accessibility, input guidance, and mobile usability.
- Add automated tests for validation, API responses, and fallback calculations.
- Strengthen production security with HTTPS, rate limiting, managed secrets, logging, and monitoring.
- Review the API-key handling flow and Ngrok launcher before production deployment.

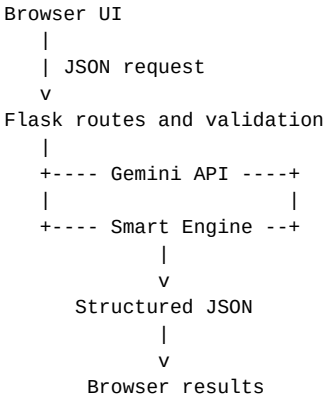
28. Educational Value

Area	Concepts demonstrated
Backend Development	Flask routing, request parsing, validation, and JSON responses.
Frontend Development	HTML templates, CSS design system, JavaScript events, and dynamic rendering.
API Integration	Calling an external generative AI service and handling service failures.
Prompt Engineering	Building structured prompts from user-provided financial data.
Algorithm Design	Implementing deterministic budget rules and fallback calculations.
Error Handling	Handling invalid requests, missing configuration, model fallback, and malformed output.
Security Basics	Environment variables, ignored secret files, input validation, and safe DOM rendering.
Version Control	Managing source code and documentation using Git and GitHub.

29. Technical Architecture Summary

The project is organised around a request-response cycle. The browser gathers a financial profile, sends JSON to Flask, and renders the returned analysis. The backend validates the request and selects the best available engine.

Layer	Implementation	Main responsibility
Presentation	HTML / CSS / JavaScript	Input form, API key interface, loading/error states, result rendering.
Application	Flask (app.py)	Routes, validation, engine selection, JSON response.
AI service	Gemini 2.0 Flash / 1.5 Flash	Generative financial analysis when configured.
Fallback logic	Smart Advisory Engine	Rule-based budget calculations and saving suggestions.
Configuration	python-dotenv / environment	Loads API and application secrets.
Temporary sharing	pyngrok	Exposes the local service through a temporary public tunnel.



30. Conclusion and References

FinanceBot demonstrates how generative AI can be combined with conventional web development to make personal budgeting information easier to understand. The application accepts income, expense categories, and a financial goal, then presents a recommended budget, spending analysis, saving suggestions, and an overall summary.

Its dual-engine design is a key part of the implementation: Gemini provides live AI-assisted analysis when configured, while the Smart Advisory Engine keeps the core workflow available without an external AI key. The stateless Flask architecture and browser-based frontend make the project straightforward to run and demonstrate.

The project provides a foundation for future improvements such as secure accounts, saved financial history, charts, goal tracking, downloadable reports, and stronger production deployment controls.

Project references

- GitHub repository: https://github.com/kunalkhyade-create/-New-Personal-Finnace-Advisor-Bot-
- Project README.md and REPORT.md in the repository.
- docs/architecture.md — System Architecture.
- docs/API_Documentation.md — API reference.
- docs/USER_GUIDE.md — End-user instructions.

- docs/database.md — Stateless data model description.
- Supplied reference document: Personal Finance Advisor.pdf (16 pages).

Disclaimer: This project is an educational budgeting assistant and should not be treated as professional financial, investment, tax, or legal advice.